



INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR
End Semester Examination 2022-23

Date of Examination: /11/23 Session(FN/AN) — Duration 3 hrs, Marks = 80

Sub No: CS60075

Sub Name: Natural Language Processing

Department/Centre/School: Computer Science and Engineering

Specific charts, graph paper, log book etc. required NO

Special Instructions (if any) ANSWER ALL questions. In case of reasonable doubt, make assumptions and state them upfront. Marks will be deducted for sketchy answers and claims without proper reasoning. All parts of a single question should be done at the same place. Keep probability values in fractional forms.

1. **HMMs**: Consider the HMM in Figure 1 that models a student's activities every hour, where during each hour period the student is in one of two possible hidden states: Studying (S) or playing Video games (V). While doing each activity the student will exhibit an observable facial expression of either Grinning (G) or Frowning (F).

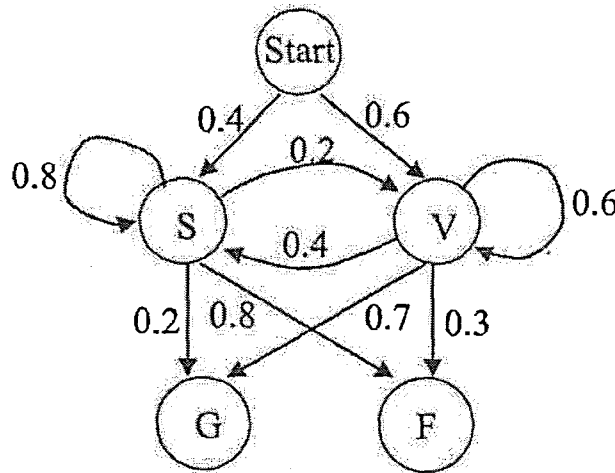


Figure 1: HMM modeling student activities.

- If in the second hour the student is Studying, what is the probability that in the fourth hour the student is playing Video games? That is, compute $P(q_4 = V | q_2 = S)$. Show the steps of computation.
- What is the probability that in the first hour the student is Grinning? That is, compute $P(o_1 = G)$. Show the steps of computation.

[4 + 4 = 8]

2. **Neural nets**: Consider a neural network with one hidden layer: $f_W(x) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}^T \sigma \left(\begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right)$ where $\sigma(z)$ is a non-linear function.

- What is the gradient $\nabla_W f_W(x)$? In particular, compute each partial derivative $\frac{\partial}{\partial W_{ij}} f_W(x)$. Leave your answer in terms of $\sigma(z)$ and $\sigma'(z) = \frac{\partial}{\partial z} \sigma(z)$.

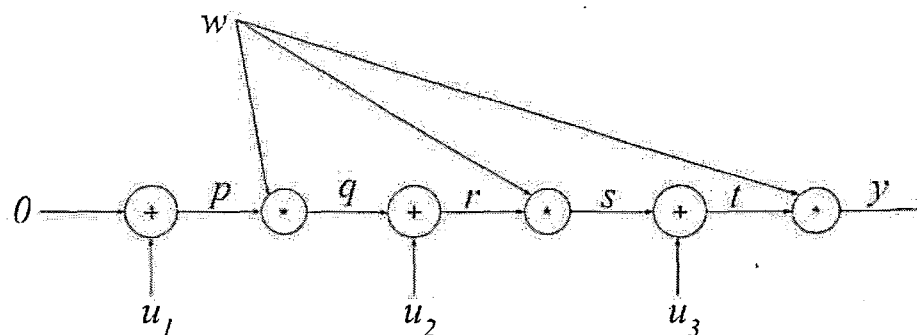


Figure 2: The 1D RNN.

- (b) What is the gradient $\nabla_W L(W; x, y)$ of the loss $L(W; x, y) = (f_W(x) - y)^2$? You do not need to expand $f_W(x)$ (but you should expand $\nabla_W f_W(x)$).

[4 + 4 = 8]

3. **Word vectors:** The following questions are in connection with the word vectors usually obtained using skip-gram/CBOW models.

- (a) Give two examples of how we can evaluate word vectors. For each example, indicate whether it is intrinsic or extrinsic.
- (b) Alice and Bob have each used the Word2Vec algorithm to obtain word embeddings for the same vocabulary of words V . In particular, Alice has obtained ‘context’ vectors $\mathbf{u}_w^A$ and ‘center’ vectors $\mathbf{v}_w^A$ for every $w \in V$, and Bob has obtained ‘context’ vectors $\mathbf{u}_w^B$ and ‘center’ vectors $\mathbf{v}_w^B$ for every $w \in V$. Suppose that, for every pair of words $w, w' \in V$, the inner product is the same in both Alice and Bob’s model: $(\mathbf{u}_w^A)^T \mathbf{v}_{w'}^A = (\mathbf{u}_w^B)^T \mathbf{v}_{w'}^B$. Does it follow that, for every $w \in V$, $\mathbf{v}_w^A = \mathbf{v}_w^B$? Why or why not?

[2 + 4 = 6]

4. **RNN theory!** Consider the following 1D RNN with no non-linearities, a 1D hidden state, and 1D inputs u_t at each timestep. (Note: There is only a single parameter w , no bias). This RNN expresses unrolling the following recurrence relation, with hidden state h_t at unrolling step t given by: $h_t = w \cdot (u_t + h_{t-1})$. The computational graph of unrolling the RNN for three timesteps is shown in Figure 2.

- (a) Fill in the blanks for the intermediate values during the forward pass, in terms of w and the u_i ’s: $p = u_1$; $q = w \cdot u_1$; $r = u_2 + q = u_2 + w \cdot u_1$; $s = w \cdot r = w \cdot u_2 + w^2 \cdot u_1$; $t = \text{_____}$; $y = \text{_____}$
- (b) Using the expression for y from the previous subpart, compute $\frac{dy}{dw}$.
- (c) Fill in the blank for the missing partial derivative of y with respect to the nodes on the backward pass. You may use values for p, q, r, s, t, y computed in the forward pass and downstream derivatives already computed. $\frac{\partial y}{\partial t} = w$; $\frac{\partial y}{\partial s} = w$; $\frac{\partial y}{\partial r} = \frac{\partial y}{\partial s} \cdot w$; $\frac{\partial y}{\partial q} = \frac{\partial y}{\partial r} \cdot 1$; $\frac{\partial y}{\partial p} = \text{_____}$
- (d) Calculate the partial derivatives along each of the three outgoing edges from the learnable w in Figure 2 (e.g., the right-most edge has a relevant partial derivative of t in terms of how much the output y is effected by a small change in w as it influences y through this edge. You need to compute the partial derivatives for the other two edges in similar lines.) You can write your answers in terms of the p, q, r, s, t and the partial derivatives of y with respect to them.
- (e) Use these three terms obtained in (d) to verify that indeed the total derivative $\frac{dy}{dw}$ in (b) is correct.

[4 + 2 + 2 + 6 + 2 = 16]

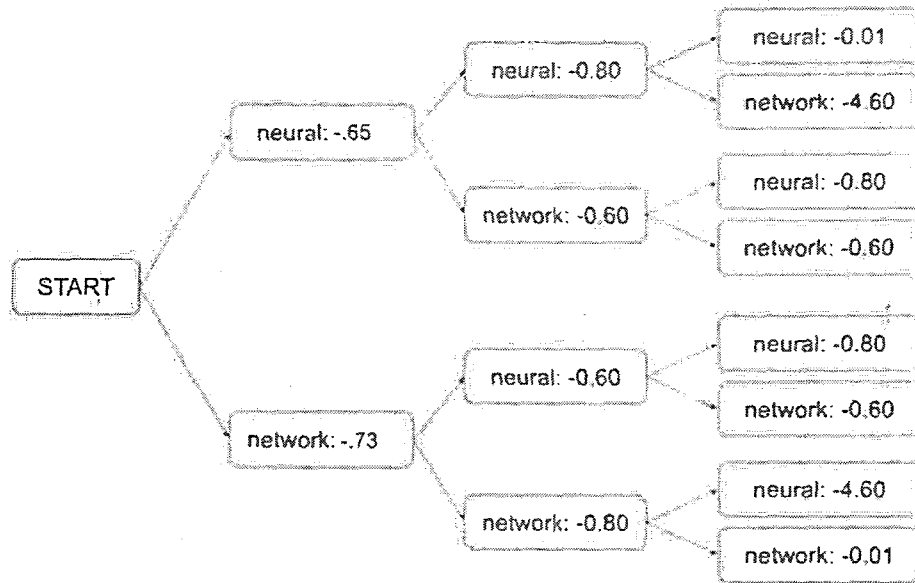


Figure 3: The numbers shown are the decoder's log probability prediction of the current token given previous tokens.

5. **Beam search:** We are running the beam search to decode a sequence of length $T = 3$ using a beam search with $k = 2$. Consider predictions of a decoder in Figure 3, where each node in the tree represents the next token log probability prediction of one step of the decoder conditioned on previous tokens. The vocabulary consists of two words: 'neural' and 'network'.
- At timestep 1, by computing scores show which sequences is beam search storing?
 - At timestep 2, by computing scores which sequences is beam search storing?
 - At timestep 3, by computing scores which sequences is beam search storing?
 - Does beam search return the overall most-likely sequence in this example? Explain why/why not.
 - Assuming that the vocabulary size is N , estimate the runtime complexity of the search algorithm in terms of T, N, k .

$$[2 + 2 + 2 + 3 + 4 = 13]$$

6. **RNN practice:** RNNs are versatile! In class, we learned that this family of neural networks have many important advantages and can be used in a variety of tasks. They are commonly used in many state-of-the-art architectures for NLP.
- For each of the following tasks, state how you would run an RNN to do that task. In particular, specify how the RNN would be used at test time (not training time), and specify $\rightarrow$ 1. how many outputs, i.e., number of times the softmax $\hat{y}^{(t)}$ is called from your RNN? (If the number of outputs is not fixed, state it as arbitrary). 2. what each $\hat{y}^{(t)}$ is a probability distribution over (e.g., distributed over all species of cats)? 3. which inputs are fed at each time step to produce each output?
 - Task 1 Named-Entity Recognition:** For each word in a sentence, classify that word as either a person, organization, location, or none. Inputs: A sentence containing n words.
 - Task 2 Sentiment Analysis:** Classify the sentiment of a sentence ranging from negative to positive (integer values from 0 to 4). Inputs: A sentence containing n words.

- iii. **Task 3** Language models: generating text from a chatbot that was trained to speak like you by predicting the next word in the sequence. Input: A single start word or token that is fed into the first time step of the RNN.

[3 + 3 + 3 = 9]

- (b) You build a sentiment analysis system that feeds a sentence into a RNN, and then computes the sentiment class between 0 (very negative) and 4 (very positive), based only on the final hidden state of the RNN.
- What is one advantage that an RNN would have over a simple window-based neural model for this task?
 - You observe that your model predicts **very positive** sentiment for the following passage: Yesterday turned out to be a terrible day. I overslept my alarm clock, and to make matters worse, my dog ate my homework. At least my dog seems happy... Why might the model misclassify the appropriate sentiment for this sentence?

[2 + 3 = 5]

7. **Transformers:** Consider the self attention module of the transformer which involves softmax. Formally, the input sequence x is projected by three matrices W_Q, W_K, W_V to corresponding representations Q, K and V respectively. Following common terminology, the Q, K and V are referred to as the 'queries', 'keys' and 'values' respectively. The output for all positions, $A_l(x) = V'$, is computed as follows, $Q = xW_Q, K = xW_K, V = xW_V$ and $A_l(x) = V' = \text{softmax}\left(\frac{QK^T}{\sqrt{D}}\right)V$. Given that subscripting a matrix with i returns the i -th row as a vector, we can write a generalized attention equation for any similarity function as follows:

$$V'_i = \frac{\sum_{j=1}^N \text{sim}(Q_i, K_j) V_j}{\sum_{j=1}^N \text{sim}(Q_i, K_j)} \quad (1)$$

For the resulting $A_l(x) = V'$, we need to compute all $V'_i \in 1, \dots, N$ where N is the sequence length.

- Assuming $\text{sim}_{\text{softmax}}(q, k) = \exp\left(\frac{q^T k}{\sqrt{D}}\right)$ and that there are no other activation functions in play we can rewrite equation 1 as _____.
- Identify the conditions that needs to be met by the sim function to ensure that V'_i in equation 1 remains finite. What constraint does this impose on the sign of the value of the function sim .
- The definition of attention in equation 1 is generic and can be used to define several other attention implementations. One potential attention variant is the 'polynomial kernel attention', where the similarity function as $\text{sim}(q, k)$ is measured by **quadratic** polynomial kernel. For degree- d polynomial the polynomial kernel is defined as $\text{sim}(x, y) = (x^T y + 1)^d$. Therefore $\text{sim}(q, k) = \text{_____}$.
- Such kernelized funtions can be written as feature maps: $\text{sim}(x, y) = \phi(x)^T \cdot \phi(y)$ where $\cdot$ is the inner product. Rewrite equation 1 with the quadratic polynomial kernel function using the feature map notation.

[2 + 3 + 2 + 2 = 9]

8. **Quest to pass!** The following (textbook) questions are in the context of transformers.

- What do you mean by teacher forcing?
- What is masking? What are the two cases where masking is useful. Illustrate with examples.

[2 + 4 = 6]